

Seguridad del Sistema

Seguridad — Sistema de IA / RAG de FastChat DJ

Documento de seguridad del sistema: **medidas ya tomadas / recomendadas** y **riesgos pendientes** con su mitigación. Se entrega con honestidad: incluye los riesgos abiertos detectados en el servidor de staging para que el cliente pueda cerrarlos antes de producción. Ningún secreto real aparece aquí; se usan **placeholders**.

1. Medidas de seguridad tomadas / recomendadas

1.1. Weaviate solo en localhost con API key

- El contenedor de Weaviate publica sus puertos **únicamente en 127.0.0.1** (127.0.0.1:8080 REST y 127.0.0.1:50051 gRPC). **No** queda expuesto a internet ni a la red pública.
- **Autenticación por API key** habilitada (`AUTHENTICATION_APIKEY_ENABLED=true`) y **acceso anónimo deshabilitado** (`AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED=false`). Solo el usuario `admin fastchatdj@local` con la API key puede operar.
- La app se conecta con la key vía `weaviate.connect_to_local(...)` (`agents_ai/weaviate_rag.py`).
- **Recomendación:** rotar `WEAVIATE_API_KEY` periódicamente y mantener el binding en `127.0.0.1` (nunca `0.0.0.0`). No abrir 8080/50051 en el firewall.

1.2. Embeddings externos (sin exponer infraestructura)

- Los embeddings del RAG se generan con **Google Gemini** (`models/gemini-embedding-001`) a través de la API oficial. No se corren modelos de embeddings locales, evitando superficie de ataque y consumo de recursos.
- Weaviate usa **vectores externos** (`vectorizer=none`): no hay módulos de vectorización dentro de Weaviate que puedan requerir salidas de red adicionales.

1.3. Credenciales en variables de entorno, no en código

- Los secretos de infraestructura del cotizador y del RAG viven en `/etc/fastchatdj.env` (`EnvironmentFile` del servicio systemd): `WEAVIATE_API_KEY` , `WEAVIATE_HOST/PORT` , `VIDANUEVA_USER/PASS` . No están hardcoded en el código.
- La configuración de la app (Postgres, `SECRET_KEY`, email, etc.) vive en `credenciales.json` (fuera de git; `credenciales_template.json` documenta las claves sin valores).
- Las **API Keys de los LLM** (Gemini/OpenAI/Claude/Ollama) se guardan en base de datos por perfil (`ApiKeyIA`) y se administran desde el panel — no en código ni en el repositorio.

- **Recomendación:** `chmod 600` y dueño restringido para `/etc/fastchatdj.env` y `credenciales.json`; nunca commitear estos archivos ni imprimir sus valores en logs.

1.4. Soft-delete (borrado lógico)

- Todos los modelos heredan de `ModeloBase` y usan borrado lógico (`status=False`), nunca `.delete()` físico. Esto evita pérdida accidental de datos y deja rastro de auditoría.
- El indexador del RAG (`indexador_conocimiento.reindexar_agente`) además **limpia los vectores huérfanos** de las fuentes marcadas como borradas (`status=False`), para que un documento “eliminado” en el panel no siga siendo recuperable por el chat.

1.5. Validación de archivos subidos

- La acción `subir_documento` (pestaña Conocimiento) valida:
 - **Extensión:** solo `pdf`, `csv`, `xlsx`, `xls`, `json`, `txt`, `docx`. Cualquier otra se rechaza.
 - **Tamaño:** máximo **20 MB**.
- **Recomendación:** el proyecto exige además `FileExtensionValidator` + validador de tamaño de `core/validadores.py` en todo `FileField`; mantener esa política en cualquier nuevo campo de archivo.

1.6. CSRF en las acciones del panel

- Las acciones administrativas del panel (subir documento, reindexar, listar modelos, guardar modelo, procesar agente) pasan por el flujo autenticado de Django con **protección CSRF** activa. Son operaciones que requieren sesión de administrador del perfil.
- **Nota:** los endpoints **públicos** del cotizador (`api_cotizar`, `api_cliente_cedula`) están marcados con `csrf_exempt` por diseño (formulario público sin sesión). Esto es aceptable porque solo **leen/consultan** (cotización y datos por cédula) y no realizan operaciones destructivas; aun así conviene protegerlos con rate-limiting (ver 2.7).

1.7. Aislamiento de conocimiento por tenant / agente

- El RAG es **multi-tenant con un tenant por agente** (`agente_{id}`). El conocimiento de un agente está **físicamente aislado** en su tenant de Weaviate; una consulta de un agente solo puede recuperar de su propio tenant (`buscar(agente_id, ...)`).
 - Esto previene fuga de información entre empresas/agentes distintos en la misma instalación multi-tenant.
 - **Recomendación:** al dar de baja un agente, borrar/vaciar su tenant para no dejar conocimiento residual.
-

2. Riesgos pendientes (detectados) y mitigación

Estos riesgos provienen del servidor de staging (2.24.107.52). **Deben cerrarse antes de un despliegue de producción.** Se listan con honestidad y con su mitigación recomendada.

2.1. 🚫 Token de GitHub expuesto en la URL del remote git

- **Riesgo:** el remote git tiene un token (`ghp_...`) en texto plano dentro de la URL (`git remote -v` lo muestra). Cualquiera con acceso al repo/servidor puede leerlo y usarlo.
- **Severidad:** Alta.
- **Mitigación:**

1. **Revocar el token inmediatamente** en GitHub (Settings → Developer settings → Personal access tokens).
2. Reconfigurar el remote sin token: usar **deploy key SSH** o un *credential helper*.

```
git remote set-url origin git@github.com:<org>/<repo>.git # SSH
# o credential helper:
git config --global credential.helper store # y usar un PAT de un solo
      uso, no en la URL
```

3. Rotar cualquier otro secreto que haya podido quedar en el historial.

2.2. 🚫 Acceso SSH como root con contraseña

- **Riesgo:** SSH permite login como `root` con contraseña (sin clave pública). Vulnerable a fuerza bruta; ya se observaron *lockouts* de sshd por exceso de conexiones.
- **Severidad:** Alta.
- **Mitigación:**
 1. Crear un usuario sudo sin privilegios y usar **claves SSH** (deshabilitar password).
 2. En `/etc/ssh/sshd_config`: `PermitRootLogin no`, `PasswordAuthentication no`.
 3. Considerar `fail2ban` y cambiar el puerto por defecto.

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
```

2.3. 🟡 `ALLOWED_HOSTS = ["*"]`

- **Riesgo:** Django acepta cualquier `Host` header → facilita ataques de *Host header injection* / cache poisoning.
- **Severidad:** Media.
- **Mitigación:** restringir a los hosts reales:

```
ALLOWED_HOSTS = ["<dominio_real>", "<ip_real>"]
```

2.4. 🚫 Sin HTTPS (tráfico en claro por HTTP :80)

- **Riesgo:** todo el tráfico (sesiones, credenciales, webhooks de WhatsApp, datos personales de cédula) viaja **sin cifrar**. Interceptable en la red.
- **Severidad:** Alta.
- **Mitigación:**
 1. Emitir certificado **Let's Encrypt** (`certbot --nginx`).
 2. Redirigir HTTP → HTTPS, activar HSTS.
 3. En Django (producción): `SECURE_SSL_REDIRECT=True` , `SESSION_COOKIE_SECURE=True` , `CSRF_COOKIE_SECURE=True` , `SECURE_PROXY_SSL_HEADER=( 'HTTP_X_FORWARDED_PROTO', 'https' )` .

2.5. 🐛 Daphne corre como root

- **Riesgo:** el proceso de la aplicación corre con privilegios de `root` ; una vulnerabilidad en la app se convierte en compromiso total del servidor.
- **Severidad:** Media-Alta.
- **Mitigación:** ejecutar el servicio con un **usuario sin privilegios** (`User=<usuario_del_servicio>` en el unit systemd), dueño de `/home/fastchatdj` y de los archivos de config. Ver el unit de ejemplo en `GUIA_INSTALACION_NODO.md` (sección A.6).

2.6. 🐛 Permisos de `credenciales.json`

- **Riesgo:** `credenciales.json` con permisos `-rwxrwxr-x` (775) → legible por grupo/otros. Contiene `SECRET_KEY` , password de Postgres, credenciales de email.
- **Severidad:** Media-Alta.
- **Mitigación:**

```
chmod 600 /home/fastchatdj/credenciales.json
chown <usuario_del_servicio>:<usuario_del_servicio> /home/fastchatdj/credenciales.json
```

Aplicar lo mismo a `/etc/fastchatdj.env` .

2.7. 🟡 Endpoints públicos del cotizador sin rate-limit

- **Riesgo:** `api_cotizar` y `api_cliente_cedula` (`csrf_exempt` , públicos) pueden ser abusados: la consulta por cédula devuelve **datos personales** (edad, sexo, nombres) y podría usarse para enumeración/scraping.
- **Severidad:** Media (privacidad).
- **Mitigación:**
 1. Aplicar **rate-limiting** por IP (nginx `limit_req` o middleware).
 2. Evaluar exigir un token/captcha en el cotizador público.

3. Minimizar los datos devueltos por la API de cédula a lo estrictamente necesario para cotizar.

2.8. 🟡 XSS potencial en el template del cotizador

- **Riesgo:** el template del cotizador usa `innerHTML` con datos propios de BD (riesgo XSS bajo, porque el contenido es propio y no de usuario, pero presente).
- **Severidad:** Baja.
- **Mitigación:** sanitizar / usar `textContent` antes de exponer el cotizador a público general.

2.9. 🟡 Fragilidad del pool de keys Gemini (embeddings)

- **Riesgo:** el motor resuelve la key Gemini **activa más nueva** del perfil. Si se agregan keys inválidas más nuevas, el RAG puede quedarse sin embeddings (incidente real de esta fase: 7 de 9 keys estaban muertas y tumbaron el RAG del bot).
- **Severidad:** Media (disponibilidad, no confidencialidad).
- **Mitigación operativa inmediata:** mantener **una sola** key Gemini válida y activa por perfil; deshabilitar (no borrar) las inválidas.
- **Mitigación de código (pendiente):** implementar **failover** de key de embeddings en `agente_consultor` e `indexador_conocimiento` (probar keys hasta encontrar una que funcione).

3. Resumen de acciones prioritarias

Prioridad	Acción	Referencia
🔴 Inmediata	Revocar el token de GitHub del remote y reconfigurar con SSH	2.1
🔴 Inmediata	Habilitar HTTPS (Let's Encrypt) y cookies seguras	2.4
🔴 Alta	Deshabilitar SSH root/password, usar claves	2.2
🟡 Alta	<code>credenciales.json</code> y <code>/etc/fastchatdj.env</code> a 600	2.6
🟡 Alta	Daphne con usuario no-root	2.5
🟡 Media	Restringir <code>ALLOWED_HOSTS</code>	2.3
🟡 Media	Rate-limit en endpoints públicos del cotizador	2.7
🟡 Media	Failover de key de embeddings Gemini	2.9
🟡 Baja		2.8

Prioridad	Acción	Referencia
	Sanitizar <code>innerHTML</code> del cotizador	

Las medidas ya implementadas (sección 1) cubren el **aislamiento del RAG**, la **protección de Weaviate**, el **manejo de secretos por entorno**, la **validación de archivos** y el **borrado lógico**. Los pendientes de la sección 2 son mayormente de **hardening de infraestructura** (SSH, HTTPS, permisos, usuario del servicio) y deben resolverse antes de exponer el sistema a producción.

4. Seguridad del widget de chat embebible

El chatbot embebible (ver `WIDGET_CHAT.md`) se diseñó para exponerse en páginas públicas de terceros sin filtrar credenciales:

Control	Implementación
Sin credenciales en el navegador	El widget solo maneja un embed key público. El <code>webservice_token</code> y la API key del proveedor LLM nunca salen del servidor.
Embed key a prueba de manipulación	Token firmado con <code>django.core.signing</code> (HMAC vía <code>SECRET_KEY</code>). Alterarlo para apuntar a otro agente invalida la firma → 403 . Verificado.
Restricción por dominio (opcional)	Se hornean dominios permitidos dentro del embed key; el proxy valida <code>Origin / Referer</code> . Recomendado en producción para cada cliente.
Rate limiting	<code>@rate_limit(limit=40, seconds=60)</code> por IP en el proxy de mensajes.
CORS controlado	El proxy responde <code>Access-Control-Allow-Origin</code> y maneja <code>OPTIONS</code> ; el JS se sirve con CORS <code>*</code> (solo lectura, sin secretos).
XSS en el render	El widget escapa el texto (<code>textContent</code>) antes de aplicar el markdown mínimo (negritas/listas/enlaces), evitando inyección de HTML desde las respuestas.
Captura de lead sin exponer datos	El lead se registra server-side (proxy) en el CRM; el navegador nunca recibe datos de otros contactos. La “sesión web” del CRM usa <code>proveedor='meta' / estado='conectado'</code> para no ser tocada por crons de reconexión, y expiración lejana para no disparar envíos de WhatsApp.

Recomendaciones de hardening del widget para producción: - Generar el embed key de cada cliente **con** `--origins` (restringido a su dominio). - Servir todo bajo **HTTPS** (el widget hereda el esquema del servidor). - Vigilar el consumo por agente con las **alertas de consumo** ya existentes.